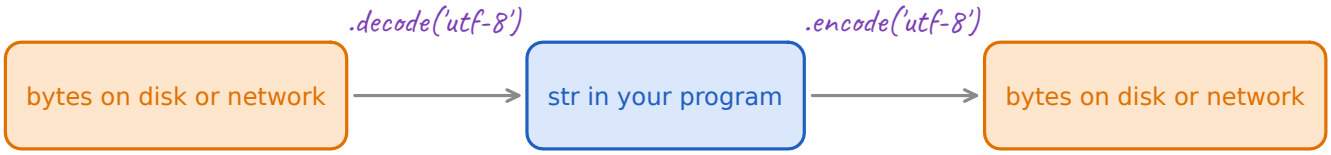
Chapter 8

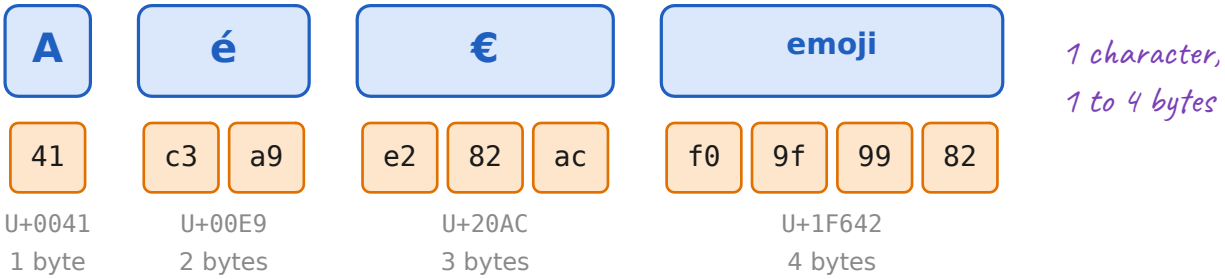
Files and data formats

a file is bytes; text is bytes plus an agreed encoding

Every file is a sequence of **bytes** (numbers 0–255). Text only exists once you agree how bytes map to characters: an **encoding**. Python keeps the two apart with two types: `bytes` for raw data, `str` for text, and you convert at the edges of your program:



UTF-8, the encoding of the web and of nearly every modern file, uses 1 byte for ASCII characters and up to 4 for everything else:




```
>>> s = "café €5"
>>> len(s), len(s.encode("utf-8"))
(7, 10)
>>> s.encode("utf-8")
b'caf\xc3\xa9 \xe2\x82\xac5'
>>> s.encode("utf-8").decode("latin-1")
'cafÃ© â\x82¬5'
>>> b"caf\xc3".decode("utf-8")
UnicodeDecodeError: 'utf-8' codec can't decode byte 0xc3 in position 3: unexpected en…
```

RULE OF THUMB

Decode as early as possible, work in `str`, encode as late as possible, and **always** pass `encoding="utf-8"` to `open()` for text. The default depends on the operating system (Windows has historically used cp1252), which is how "works on my machine" bugs start.

open(), modes and pathlib

mode	means	if the file exists	if it doesn't
`"r"`	read text (default)	read from the start	`FileNotFoundError`
`"w"`	write text	**emptied first**	created
`"a"`	append text	write at the end	created
`"x"`	exclusive create	`FileExistsError`	created
add `b`	bytes, not str (`"rb"`, `"wb"`)	no encoding, no newline translation	
add `+`	read and write (`"r+"`)	rarely what you want	

```python
from pathlib import Path

data = Path("data")
data.mkdir(exist_ok=True)
notes = data / "notes.txt"                          # / joins paths on any OS

notes.write_text("line one\nline two\n", encoding="utf-8")
with notes.open("a", encoding="utf-8") as fh:
    fh.write("line three\n")

with notes.open(encoding="utf-8") as fh:
    for n, line in enumerate(fh, 1):                # one line at a time: any size of file
        print(n, line.rstrip("\n"))

print(notes.name, notes.stem, notes.suffix, notes.parent, notes.stat().st_size, "bytes")
print(sorted(p.name for p in data.glob("*.txt")))
```

```
1 line one
2 line two
3 line three
notes.txt notes .txt data 29 bytes
['notes.txt']
```

pathlib	instead of
`Path.home() / "docs" / "a.txt"`	`os.path.join(os.path.expanduser("~"), ...)`
`p.exists()`, `p.is_file()`, `p.is_dir()`	`os.path.exists(p)` ...
`p.read_text()`, `p.write_bytes(b)`	open / read / close by hand
`p.glob("**/*.csv")`, `p.iterdir()`	`glob.glob`, `os.listdir`
`p.with_suffix(".bak")`, `p.resolve()`	string slicing on paths

TRAP

`for line in fh` keeps the `\n` on every line. Use `line.rstrip("\n")` (or `.strip()` if surrounding spaces don't matter). And `fh.read()` loads the whole file into memory: fine for kilobytes, not for a 4 GB log.

CSV: tables as text

CSV looks trivial until a field contains a comma, a quote or a newline. `line.split(",")` breaks on all three; the `csv` module handles the quoting rules:

```
import csv
from pathlib import Path

rows = [{"title": "Dune", "author": "Herbert, Frank", "note": 'says "spice"'},
        {"title": "Emma", "author": "Austen, Jane", "note": "two\nlines"}]

with open("books.csv", "w", newline="", encoding="utf-8") as fh:
    w = csv.DictWriter(fh, fieldnames=["title", "author", "note"])
    w.writeheader()
    w.writerows(rows)

print(Path("books.csv").read_text(encoding="utf-8"))

with open("books.csv", newline="", encoding="utf-8") as fh:
    back = list(csv.DictReader(fh))
print(back == rows, back[0]["author"])
```

```
title,author,note
Dune,"Herbert, Frank","says ""spice"""
Emma,"Austen, Jane","two
lines"

True Herbert, Frank
```

line 8: newline="": let csv handle line ends

split by hand (broken)

```
line = '"Herbert, Frank",Dune'
print(line.split(","))
```

```
['"Herbert', ' Frank"', 'Dune']
```

csv.reader

```
import csv
print(next(csv.reader(['"Herbert, Frank",Dune'])))
```

```
['Herbert, Frank', 'Dune']
```

TRAP

Everything from CSV comes back as a **string**: `"1965"`, not `1965`, and an empty field is `""`, not `None`. Convert and validate each column yourself (or use pandas, which guesses types for you, sometimes wrongly).

JSON: nested data as text

JSON stores nested lists and dicts, and it's what web APIs speak (Ch 15). It has fewer types than Python, so some things change on the round trip:

Python	JSON	comes back as
`dict` (str keys)	object `{...}`	`dict`
`dict` with int keys `{1: "a"}`	`{"1": "a"}`	**str keys!**
`list`, `tuple`	array `[...]`	`list` (tuples are lost)
`str`, `int`, `float`	string, number	same
`True`, `False`, `None`	`true`, `false`, `null`	same
`set`, `datetime`, `Decimal`, your classes	**not allowed**	`TypeError` unless you convert

```python
import json, datetime
from dataclasses import dataclass, asdict

@dataclass
class Loan:
    title: str
    due: datetime.date

def to_jsonable(obj):                              # called for anything json can't encode
    if isinstance(obj, datetime.date):
        return obj.isoformat()
    raise TypeError(f"can't encode {type(obj).__name__}")

data = {"member": "ana", "loans": [asdict(Loan("Dune", datetime.date(2026, 10, 3)))], 7: (1, 2)}
text = json.dumps(data, default=to_jsonable, indent=2)
print(text)
print(json.loads(text)["7"])
```

```
{
  "member": "ana",
  "loans": [
    {
      "title": "Dune",
      "due": "2026-10-03"
    }
  ],
  "7": [
    1,
    2
  ]
}
[1, 2]
```

KEY IDEA

For logs and big datasets use **JSON Lines**: one JSON object per line. You can append without rewriting the file, and read it line by line: `[json.loads(l) for l in fh]`.

TOML, pickle, and choosing a format

```python
import tomllib                    # read-only TOML parser, in the stdlib since 3.11

config = tomllib.loads('''
[library]
name = "Central"
max_loans = 3
open_days = ["Mon", "Wed", "Fri"]
''')
print(config["library"]["max_loans"] + 1, config["library"]["open_days"][-1])
```

```
4 Fri
```

`pickle` can save almost any Python object, but loading a pickle can run **arbitrary code**: the file itself says which functions to call while rebuilding objects. This harmless example runs `print`; a malicious one would run `os.system`:

```python
import pickle

class Evil:
    def __reduce__(self):                    # "to rebuild me, call print(...)"
        return (print, ("this ran just by loading the file!",))

payload = pickle.dumps(Evil())
pickle.loads(payload)
```

```
this ran just by loading the file!
```

TRAP

Never unpickle data from anyone you don't trust: not uploads, not downloads, not network messages. Use JSON for data exchange. Pickle is for your own caches, read only by your own program.

format	good for	watch out
CSV	flat tables, spreadsheets	everything is a string; quoting
JSON / JSON Lines	nested data, APIs, logs	no dates, sets or int keys
TOML	config files written by humans	stdlib can read but not write it
SQLite (Ch 12)	data you query and update	a schema to design
pickle	caching your own objects	**runs code when loaded**
binary (`struct`)	fixed file formats, protocols	byte order, versioning

Writing files safely

If the program crashes or the power fails half-way through `write_text`, the file is left truncated: the old data is gone and the new data is incomplete. The fix is to write a **temporary file** next to the target, then swap it in with `os.replace`, which is atomic (it either fully happens or not at all):




```python
import json, os, tempfile
from pathlib import Path

def write_atomic(path, text):
    path = Path(path)
    fd, tmp = tempfile.mkstemp(dir=path.parent, prefix=path.name, suffix=".tmp")
    try:
        with os.fdopen(fd, "w", encoding="utf-8") as fh:
            fh.write(text)
            fh.flush()
            os.fsync(fh.fileno())          # make sure the bytes reached the disk
        os.replace(tmp, path)              # atomic swap on the same filesystem
    except BaseException:
        os.unlink(tmp)
        raise

write_atomic("members.json", json.dumps({"ana": 20}))
print(Path("members.json").read_text(), [p.name for p in Path(".").glob("*.tmp")])
```

```
{"ana": 20} []
```

Binary formats: struct

Many file formats start with a fixed binary header. `struct` packs numbers into exact bytes; the format string gives the byte order and the size of each field:

```python
import struct

header = struct.pack("<4sHI", b"LIBR", 2, 1_000_000)   # little-endian: 4 bytes, uint16, uint32
print(header, len(header))
magic, version, count = struct.unpack("<4sHI", header)
print(magic, version, count)
print(struct.pack(">I", 1).hex(), struct.pack("<I", 1).hex())    # big vs little endian
```

```
b'LIBR\x02\x00@B\x0f\x00' 10
b'LIBR' 2 1000000
00000001 01000000
```

IN OTHER LANGUAGES

Every language has the same pair: a text layer (encoding, newline handling) over a byte layer. Java has `Reader`/`InputStream`, C has `fopen("r")`/`fopen("rb")` (the distinction only matters on Windows), Rust has `String` vs `Vec<u8>` and refuses invalid UTF-8 in a `String`.

Compression, archives and copying

```python
import gzip, shutil, zipfile
from pathlib import Path

Path("log.txt").write_text("GET /book/3\n" * 10_000, encoding="utf-8")
with open("log.txt", "rb") as src, gzip.open("log.txt.gz", "wb") as dst:
    shutil.copyfileobj(src, dst)                    # streams in chunks: no full read
print(Path("log.txt").stat().st_size, "->", Path("log.txt.gz").stat().st_size, "bytes")

with zipfile.ZipFile("backup.zip", "w", zipfile.ZIP_DEFLATED) as z:
    z.write("log.txt")
    z.writestr("readme.txt", "made by Python")
print(zipfile.ZipFile("backup.zip").namelist())

shutil.make_archive("snapshot", "gztar", root_dir=".", base_dir="data")
print(Path("snapshot.tar.gz").exists())
```

```
120000 -> 295 bytes
['log.txt', 'readme.txt']
True
```

module	for
`shutil`	copy, move, delete trees (`rmtree`), archives, disk usage
`gzip`, `bz2`, `lzma`, `zlib`	compress one stream; `compression.zstd` is new in 3.14
`zipfile`, `tarfile`	archives of many files (beware `../` paths when extracting untrusted ones)
`tempfile`	temporary files and directories that clean themselves up

Practise, then stamp

NOW PRACTISE (practice/ folder, no installs needed)

Run `python -m practice check 8.1` (then the next id). Tests check what your code **does**, not how it looks, so any correct solution passes. After a pass, `python -m practice alt 8.1` shows other valid solutions and their trade-offs.

8.1 build from a spec CSV that survives awkward names
8.2 fix the bug Counting characters, not bytes
8.3 build from a spec JSON that keeps its types
8.4 build from a spec Saving without ever corrupting the file
8.5 build from a spec A binary file format with struct
8.6 mini-project Mini-project: summarise a folder of logs

PASSPORT STAMP ask the same question of every new language (Appendix B)

10. I/O

Files are bytes; `str` is decoded text. `open(path, mode, encoding="utf-8")` in a `with`; `pathlib` for paths; `csv`, `json`, `tomllib`, `struct`, `gzip`/`zipfile` for formats. Atomic writes via temp file + `os.replace`.

12. Stdlib and tooling

Stdlib covers the common formats; never unpickle untrusted data.

DEEP DIVE (your vault)

Vault notes: 5.12 Bytes and the text boundary · 11.2 Binary and text I/O · 11.3 File objects, modes, and offsets · 11.5 Paths and filesystem operations · 11.6 Buffers, flushing, and durability · 11.7 JSON · 11.8 CSV · 11.9 Pickle and object reconstruction · 11.15 pathlib · 11.16 Temporary resources · 11.17 Integrated serialisation pipeline.